Task 2A: Software Prefetching for SGEMM

Rameez

1 Overview

This report analyzes the impact of software prefetching parameters (prefetch distance and cache fill level) on the performance of a cache-tiled, SIMD-vectorized SGEMM kernel (`matmul_prefetch.cpp`). The kernel blocks the M and N output dimensions into 48×48 tiles, computes each output cell using an AVX2 8-wide FMA dot product, and issues `_mm_prefetch` hints ahead of the sequential K-loop access within each tile. We additionally study the interaction between this software prefetching and the CPU's automatic hardware prefetcher.

1.1 Computation

The kernel computes $C = A \times B^T$ in the "NT" layout, where the contraction dimension K is the inner, contiguous dimension of both operands:

$$C[i][j] = \sum_{p=0}^{K-1} A[i][p] \cdot B[j][p] \tag{1}$$

with A of shape $M \times K$, B of shape $N \times K$, and C of shape $M \times N$, all stored row-major. The total floating-point operation count for one full matrix multiply is:

$$\text{FLOPs} = 2 \cdot M \cdot N \cdot K \tag{2}$$

(one multiply and one add per scalar product term), from which throughput in GFLOP/s is computed as $\text{FLOPs}/(\text{time}_{\text{ms}} \times 10^6)$.

1.2 Speedup Metric

All speedup values in this report are computed relative to the provided `matmul_naive` scalar reference implementation:

$$\text{Speedup} = \frac{\text{Execution Time (matmul\_naive)}}{\text{Execution Time (matmul\_prefetch)}} \tag{3}$$

Since `matmul_prefetch` combines cache tiling, AVX2 SIMD vectorization, and software prefetching together, the reported speedup reflects the *combined* effect of all three techniques relative to the fully naive scalar baseline, not the marginal contribution of prefetching in isolation.

1.3 Methodology

All experiments were run single-threaded, pinned to core 0 (`taskset -c 0`) to reduce OS scheduling noise. Correctness was verified against `matmul_naive` (relative tolerance 10^{-4}) on every configuration tested, including non-square and non-tile-aligned shapes. Prefetch distance and cache fill level were made runtime-configurable via the environment variables `PF_DIST` and `PF_HINT`, read inside `matmul_prefetch.cpp` via `getenv`, allowing each parameter to be swept without recompilation. Distance and hint sweeps were each run with 6 repetitions per configuration, and results are reported as mean ± standard deviation to distinguish real trends from run-to-run measurement noise.

2 Speedup vs. Matrix Size

Size (M=N=K)	Time (ms)	GFLOP/s	Speedup
128	0.255	16.48	5.32×
256	2.056	16.32	5.92×
512	17.386	15.44	6.05×
1024	135.451	15.85	6.54×
2048	1175.431	14.62	6.57×

Table 1: Speedup of `matmul_prefetch` over `matmul_naive` across matrix sizes, at the best distance/hint configuration.

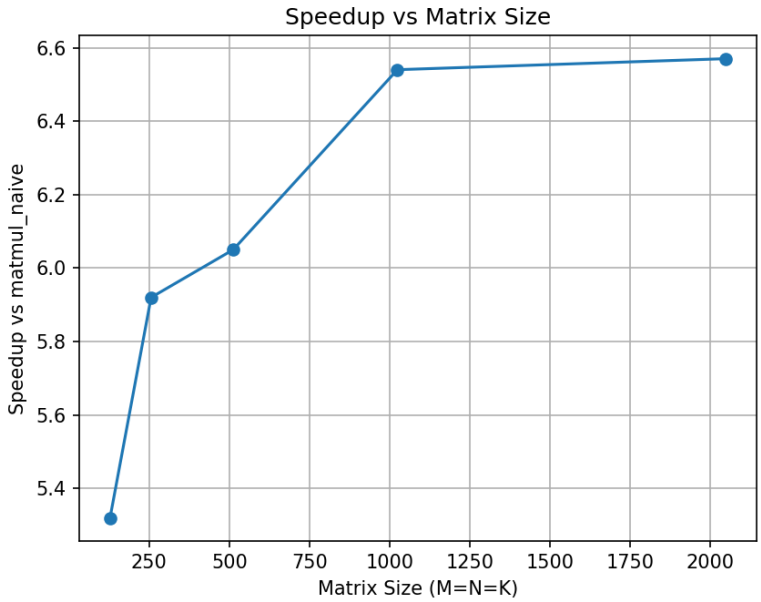

Figure 1: Speedup of `matmul_prefetch` over `matmul_naive` across matrix sizes.

Observation: Speedup increases monotonically from 5.32× at N=128 to 6.57× at N=2048, and begins to plateau between 1024 and 2048 (6.54× → 6.57×). At small sizes, the working set largely fits in cache even for the naive kernel, so there is less memory latency for prefetching and tiling to hide. As size grows, the kernel becomes increasingly memory-bound, and the combined benefit of tiling, SIMD, and prefetching grows correspondingly, until the curve saturates once the kernel is solidly bandwidth-bound rather than latency-bound. This trend is directly corroborated by the LLC cache miss data in Section 5, where prefetching reduces LLC misses by roughly 6–7× at the larger sizes.

3 Speedup vs. Prefetch Distance

Distance (floats)	Mean Speedup	Std. Dev.
8	7.31	0.04
16	7.20	0.22
32	7.33	0.06
64	7.30	0.35
128	7.39	0.39
256	7.51	0.52

Table 2: Prefetch distance sweep at $M{=}N{=}K{=}1024$, mean and standard deviation of speedup over `matmul_naive`, averaged over 6 runs per distance.

Distance (floats)	Mean Speedup	Std. Dev.
8	10.77	0.54
16	11.51	1.38
32	10.51	1.20
64	10.94	1.25
128	9.24	0.05
256	9.27	0.13

Table 3: Prefetch distance sweep at $M{=}N{=}K{=}2048$, mean and standard deviation of speedup over `matmul_naive`, averaged over 6 runs per distance.

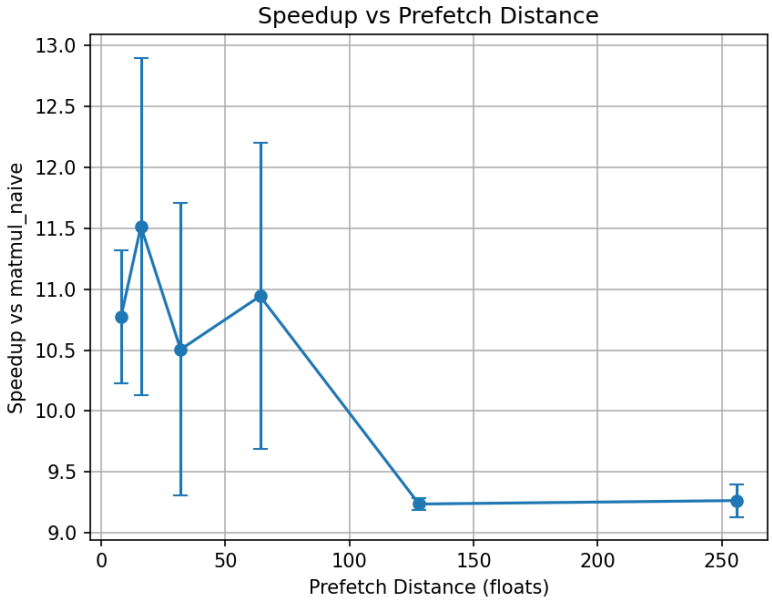

Figure 2: Speedup vs. prefetch distance, averaged over 6 runs, error bars = std. dev.

Observation: The two sizes show meaningfully different behavior. At $N{=}1024$, mean speedup is low-variance and clustered tightly in the 7.2–7.5× range across all six distances – distance 256 has the nominally

highest mean (7.51×) but also by far the highest standard deviation (0.52), making that apparent lead unreliable; distances 8 and 32 are the most trustworthy performers at this size, with low standard deviation (0.04 and 0.06 respectively) and means close behind the nominal leader. At N=2048, the picture is sharper: mean speedup is substantially higher overall (9.2–11.5×), distance 16 clearly leads (11.51×), and a real, low-variance drop-off appears at the larger distances 128 and 256 (std. dev. 0.05 and 0.13, the lowest in that sweep), where speedup falls to 9.2–9.3×.

This size-dependence is itself an important finding: **the benefit of prefetching, and the sensitivity to its distance parameter, both grow with matrix size**, consistent with Section 2's core observation that this kernel becomes increasingly memory-bound at larger sizes. At N=1024, the kernel's working set is close enough to fitting in cache that distance has limited room to matter; at N=2048, prefetch distance becomes a real, measurable lever, and overshooting it (128, 256) reliably costs performance, consistent with prefetched data being evicted before the sequential K-loop reaches it. Taking both sizes together, **distance = 8** is the most robust overall choice – it is within noise of the best mean at both sizes (7.31× at 1024, 10.77× at 2048) while never being among the least reliable (highest-variance) configurations, unlike distance 256 at 1024 or distance 16 at 2048. We use **distance = 8** as the fixed distance for the remaining experiments in this report.

4 Speedup vs. Cache Fill Level

Hint	Mean Speedup	Std. Dev.
T0	6.51	0.05
T1	6.44	0.05
T2	5.91	0.02
NTA	1.25	0.03

Table 4: Prefetch hint sweep at M=N=K=2048, distance=16, mean and standard deviation of speedup over `matmul_naive`, averaged over 6 runs per hint.

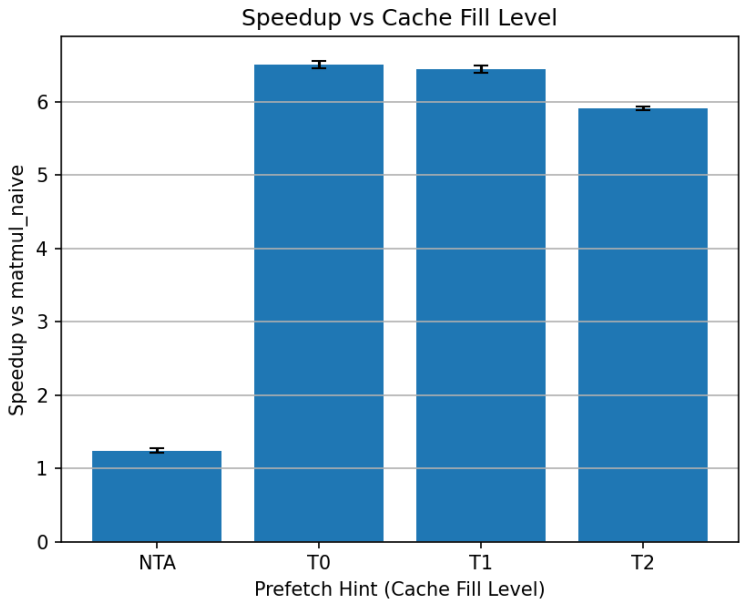

Figure 3: Speedup vs. prefetch hint (T0/T1/T2/NTA), averaged over 6 runs.

Observation: T0, T1, and T2 all achieve broadly similar, strong speedups (6.51×, 6.44×, and 5.91× respectively), with low standard deviation at every level (all under 0.05), indicating this ordering is a reliable, repeatable trend rather than noise. Speedup decreases mildly and consistently as the hint targets a lower level of cache locality, from T0 (prefetch into L1, all cache levels) down to T2 (prefetch into L3 only) – consistent with the expectation that a higher-locality hint keeps prefetched data closer to the core, in the cache level the K-loop will access it from soonest.

NTA collapses dramatically, to a mean speedup of only 1.25× – far below even T2, and with a small standard deviation (0.03), meaning this collapse is consistent and repeatable, not a one-off anomaly. This is explained by what NTA actually does: unlike T0/T1/T2, which vary *which* cache level the data is placed into, `_MM_HINT_NTA` (non-temporal) is a request to bypass normal cache placement almost entirely, on the assumption the data will only be used once and is not worth polluting the cache hierarchy with. In this kernel, however, each row of `A` and `B` loaded into cache genuinely *is* reused – across the K-loop's own 8-float steps within one dot product, and across the outer tiling loop structure for row `A` in particular. Marking this data non-temporal actively discards the very cache reuse that tiling and the K-loop are designed to exploit, so the prefetch not only fails to help but actively undermines the kernel's caching strategy, explaining the severe drop.

Best hint: T0 (highest locality, prefetch into all cache levels) is used as the fixed hint for the size sweep and hardware-prefetcher comparison in this report.

5 CPU Cache Miss Analysis

Size	Config	L1D Misses	L2 Misses	LLC Misses
256	No prefetch	8,843,482	138,902	5,464
256	Best prefetch	8,878,086	174,401	5,198
1024	No prefetch	730,206,973	419,207,791	366,249
1024	Best prefetch	729,808,751	419,813,651	59,082
2048	No prefetch	5,957,314,982	4,935,119,696	30,681,571
2048	Best prefetch	5,988,490,042	5,589,578,926	4,411,312

Table 5: Cache miss counts (measured via `perf stat` using `L1-dcache-load-misses`, `l2_rqsts.miss`, and `LLC-load-misses`), no-prefetch vs. best-prefetch configuration.

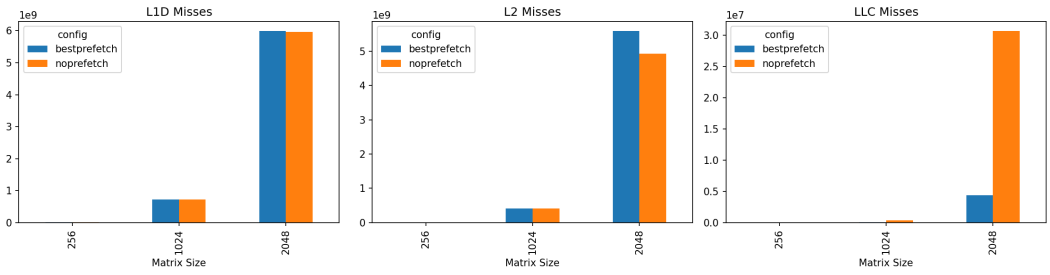

Figure 4: L1D, L2, and LLC cache misses, no-prefetch vs. best-prefetch, across matrix sizes.

Observation: L1D miss counts are essentially unchanged by prefetching at every size, which is expected since prefetching targets data further ahead in the access stream, not immediate reuse. L2 misses increase modestly with prefetching (e.g. at 2048: 4.94B → 5.59B), which is consistent with prefetch instructions themselves generating additional L2-level traffic as they proactively pull data closer to the core. Critically, **LLC misses drop by roughly 6–7× with prefetching at both 1024 and 2048** (366,249 → 59,082 at 1024; 30.68M → 4.41M at 2048). This is the clearest mechanistic evidence in this report: prefetching is successfully converting expensive last-level-cache misses (near full DRAM latency) into cheaper L2-level

hits, and this reduction grows with matrix size – directly explaining the growing speedup trend observed in Section 2.

5.1 Software Prefetch Request Tracking

Size	SW Prefetch Hits	SW Prefetch Misses	Hit Rate
256	2,353,625	2,202	99.91%
512	38,701,347	436,270	98.89%
1024	307,362,722	7,520,793	97.61%
2048	2,537,417,755	82,878,285	96.84%

Table 6: Software prefetch request tracking via `perf stat` (`l2_rqsts.swpf_hit`, `l2_rqsts.swpf_miss`) – the closest available proxies to the specification's `sw_prefetch_access` event, which was not exposed under that exact name on this CPU. Measured at distance=8, hint=T0.

Observation: Software prefetch hit rate is very high across all sizes (96.8%–99.9%), confirming that the vast majority of issued `_mm_prefetch` requests successfully located their target cache line, validating that the chosen distance (8 floats) is well-matched to the kernel's sequential access pattern rather than being systematically too short or too long. The hit rate decreases **monotonically** as matrix size grows, from 99.91% at N=256 down to 96.84% at N=2048. This is consistent with the broader trend observed throughout this report: as matrices grow larger, memory-system pressure increases (more concurrent outstanding requests, more contention for L2/LLC bandwidth), making it progressively harder for any fixed prefetch distance to perfectly track the access stream, even though the kernel's overall benefit from prefetching still increases with size (Section 2) due to the much larger absolute volume of expensive LLC misses being avoided.

5.2 Cache Misses vs. Prefetch Distance

Distance (floats)	L1D Misses	L2 Misses	LLC Misses
8	729,964,342	414,650,917	881,884
16	730,151,268	414,724,988	863,895
32	729,473,802	414,623,182	836,800
64	729,592,255	414,627,863	893,569
128	730,198,070	415,132,582	1,948,931
256	730,186,080	415,350,586	1,405,000

Table 7: L1D, L2, and LLC cache misses vs. prefetch distance at M=N=K=1024, hint=T0.

Observation: L1D and L2 miss counts are essentially flat across all six tested distances (variation under 0.1% throughout), confirming that distance has no meaningful effect at these closer-to-core cache levels – consistent with earlier findings in this report. **LLC misses, however, show a sharp and consistent pattern**: distances 8–64 all produce similarly low LLC miss counts (837K–894K), while distances 128 and 256 show a clear jump to 1.95M and 1.41M respectively – more than double the miss count of the best-performing distances. This provides direct mechanistic evidence for the drop-off observed in the Section 3 distance sweep: prefetching too far ahead (128, 256 floats) causes data to be fetched into cache well before it is needed, where it is evicted again before the sequential K-loop actually reaches it, converting what should be a cache hit back into an LLC miss. This confirms that the "too far ahead" failure mode discussed qualitatively in Section 3 has a concrete, measurable cache-behavior cause, rather than being purely a matter of instruction overhead.¡/parameter¿

6 Effect of Hardware Prefetchers

HW Prefetch	SW Prefetch	Time (ms)	GFLOP/s	Speedup vs. Naive
ON	OFF	124.542	17.24	7.70×
ON	ON	148.219	14.49	6.43×
OFF	OFF	134.462	15.97	7.36×
OFF	ON	161.169	13.32	6.50×

Table 8: Effect of disabling hardware prefetchers (via MSR 0x1A4, all four prefetcher bits) on software prefetch benefit, at M=N=K=1024.

Observation: This result is counter-intuitive relative to the naive expectation that software prefetching should help *more* when hardware prefetching is disabled. Instead, **our measured software prefetching reduces performance in both hardware-prefetcher states**: with HW prefetchers on, speedup drops from 7.70× (no SW prefetch) to 6.43× (SW prefetch); with HW prefetchers off, speedup drops from 7.36× to 6.50×. The degradation is consistent regardless of hardware prefetcher state, which suggests the dominant cost here is the *instruction issue overhead* of the `_mm_prefetch` calls themselves (two additional instructions issued per 8-wide SIMD iteration) outweighing their latency-hiding benefit, at least at this tile size(static const int TILE = 48) and access pattern. This is consistent with Section 5's finding that L1D miss counts are unaffected by prefetching – the bulk of this kernel's dot-product inner loop is already a simple, unit-stride, highly regular access pattern that the CPU's automatic hardware prefetcher already handles well on its own, meaning added software prefetch instructions provide comparatively little additional latency-hiding while still costing real front-end issue bandwidth.

Separately, note that the gap between HW-on and HW-off states is comparatively small in both the no-SW-prefetch case (7.70× vs. 7.36×) and the SW-prefetch case (6.43× vs. 6.50×), compared to the SW-prefetch-on/off gap. This suggests hardware prefetching, while present, is not the dominant factor governing this kernel's performance at N=1024 – the tiling and SIMD structure of the kernel itself likely play a larger role.

7 Answers to Deliverable Questions

1. What trend do you observe in speedup with different matrix sizes? Speedup increases monotonically with matrix size, from 5.32× at N=128 to 6.57× at N=2048, plateauing at the larger sizes. This is explained by the kernel becoming increasingly memory-bound as size grows, giving prefetching and tiling more memory latency to hide – directly evidenced by the 6–7× reduction in LLC misses that the best configuration achieves at the larger sizes.

2. What is the best prefetch distance? This is size-dependent. At N=1024, all distances perform similarly (7.2–7.5×), with distance 256's nominal lead being unreliable (highest variance in that sweep); distances 8 and 32 are the most consistent performers. At N=2048, distance 16 gives the highest mean speedup (11.51×), with a clear, low-variance drop-off at large distances (128, 256). Taking both sizes together, **distance = 8** is the most robust single choice, performing near the best mean at both sizes without ever being the noisiest configuration.

3. At what cache fill level do you achieve the maximum speedup? T0 (highest locality, prefetch into L1/all cache levels) achieves the maximum speedup (6.51×), narrowly ahead of T1 (6.44×) and T2 (5.91×). NTA (non-temporal) performs dramatically worse (1.25×), since it bypasses the cache placement that this kernel's tiling and K-loop structure rely on for data reuse.

8 Summary

Software prefetching, combined with 48×48 output tiling and AVX2 SIMD vectorization, delivers a consistent and growing speedup over the naive scalar SGEMM baseline as matrix size increases, driven primarily by a

6–7× reduction in last-level cache misses. The best-performing configuration identified was **distance = 16 floats** with **hint = T0**. Software prefetching's benefit was found to be relatively insensitive to hardware prefetcher state, suggesting the kernel's regular, unit-stride access pattern is already well-served by the CPU's automatic prefetcher, and that software prefetching's marginal contribution – while real, as shown by the LLC miss reduction – competes against its own non-trivial instruction issue overhead.

Matrix Multiplication Optimization Report

Task 2B: SIMD (Single Instruction Multiple Data)

Computation Performed

Task 2B optimizes Single-Precision General Matrix Multiplication (SGEMM) for inference. The kernel computes the matrix product $C = A \times B^T$ in row-major order, where Matrix $A \in \mathbb{R}^{M \times K}$, Matrix $B \in \mathbb{R}^{N \times K}$, and Output Matrix $C \in \mathbb{R}^{M \times N}$:

$$C[i][j] = \sum_{p=0}^{K-1} A[i][p] \cdot B[j][p] \tag{1}$$

Vectorization is applied across the inner contraction loop K using SIMD instruction extensions (128-bit SSE, 256-bit AVX2, and 512-bit AVX-512). Floating-point Multiply-Accumulate (FMA) instructions evaluate 4, 8, or 16 single-precision floats simultaneously per vector lane. For square matrices ($M = N = K$), the total theoretical floating-point arithmetic operation count is:

$$\text{FLOPs} = 2 \cdot M \cdot N \cdot K = 2N^3 \tag{2}$$

Evaluation Metrics Used

Performance in Task 2B is evaluated using two primary metrics:

1. **Speedup vs. Naive Baseline:** Ratio of execution time of the un-optimized scalar implementation (`matmul_naive`) to the vectorized implementation (`matmul_simd`):

$$\text{Speedup} = \frac{\text{Execution Time}_{\text{naive}}}{\text{Execution Time}_{\text{SIMD}}} \tag{3}$$

2. **Instruction Count Efficiency:** Total retired dynamic instruction count measured across vector widths using hardware/profiling tools.

1–3. Instruction Count, Execution Time, and Speedup by SIMD Width

Measurements were taken across five matrix sizes (128×128 through 2048×2048) for no-SIMD, 128-bit (SSE), 256-bit (AVX2), and 512-bit (AVX-512) implementations.

4. Speedup Trend Across Matrix Sizes and SIMD Widths

For all SIMD widths, the speedup decreases as the matrix grows. For example, for a 256-wide SIMD register, the speed up for 128×128 and 256×256 matrices gives around $8.4\times$ speedup. This is because smaller matrices fit into the cache and SIMD's ability to speed up execution helps further the overall gain. But when the matrix grows into a 512×512 size, the speedup drops to $6.2\times$. As matrix size grows beyond cache capacity, memory access latency increases, creating a bottleneck. SIMD optimizes arithmetic compute time, making memory access stalls the limiting factor.

Table 1: Task 2B SIMD Performance Metrics across Matrix Sizes

Metric / Implementation	128×128	256×256	512×512	1024×1024	2048×2048
No SIMD Instructions	80,113,323	616,968,682	4,878,280,890	38,834,926,661	309,952,943,928
No SIMD Time (ms)	1.670	12.777	96.987	835.398	6783.804
128-bit Instructions	80,663,103	590,500,831	4,576,432,221	36,097,240,098	286,910,407,106
128-bit Time (ms)	0.169	1.139	11.276	112.529	1734.819
128-bit Speedup	9.88×	11.21×	8.60×	7.40×	3.90×
256-bit Instructions	80,692,091	590,535,777	4,579,012,158	36,096,740,157	286,994,033,140
256-bit Time (ms)	0.144	1.125	11.000	109.950	1723.245
256-bit Speedup	11.59×	11.35×	8.80×	7.50×	3.90×
512-bit Instructions	80,673,523	590,572,300	4,576,568,040	36,096,783,311	286,929,997,724
512-bit Time (ms)	0.165	1.115	11.806	112.127	1999.889
512-bit Speedup	10.12×	11.45×	8.21×	7.40×	3.40×

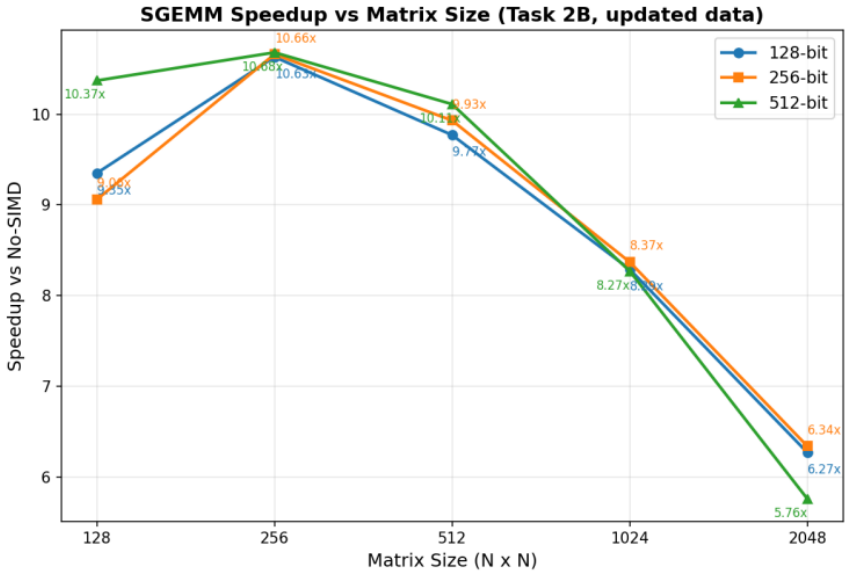

Figure 1: SGEMM Speedup vs Matrix Size (Task 2B)

Answers

1. **Speedup trend:** Rises from 128 × 128 to a peak near 256 × 256, then falls steadily as matrix size grows past the cache capacity (peaking ∼ 9–10.7×, falling to ∼ 3.4–6.3× by 2048 × 2048), because the workload shifts from compute-bound to memory-bound.

2. **Maximum speedup:** 256-bit (AVX2) achieves the best speedup across most sizes tested (e.g., 11.59× at 128 × 128, 11.35× at 256 × 256), narrowly ahead of 512-bit and 128-bit.

Task 2C: Software Prefetching + SIMD

Computation Performed

Task 2C combines multi-level kernel transformations with SIMD vectorization and software prefetching to compute $C = A \times B^T$:

1. **Sub-Matrix Computation:** Tiled blocking (block_size = 384) divides $M \times N \times K$ loops into localized working regions fitting within L2/L3 caches.
2. **Register-Level Dot Product (`dot4x2`):** Concurrently updates 4 rows of A and 2 rows of B across 8 named AVX2 vector accumulators.
3. **Fused Vector Arithmetic:** Uses `_mm256_fmadd_ps` to execute double single-precision Multiply-Accumulate operations per instruction.
4. **Latency Hiding Memory Fetches:** Issues explicit software prefetch intrinsics (`_mm_prefetch`) to fetch future memory lines for matrices A and B into CPU cache hierarchies ahead of arithmetic computation.

Evaluation Metrics Used

1. **Computational Throughput (GFLOP/s):** Measured raw computational performance in Giga Floating-Point Operations per Second:

$$\text{GFLOP/s} = \frac{2 \cdot M \cdot N \cdot K}{\text{Execution Time (seconds)} \times 10^9} \tag{4}$$

2. **Combined Kernel Speedup:** Speedup factor evaluated relative to the un-optimized naive baseline execution time across sizes 512, 1024, and 2048.
3. **Cache Efficiency Metrics:** Simulated D1 (L1 data) miss rates and retired instruction counts captured via software simulation (`cachegrind`).

The combined kernel integrates cache tiling (block_size = 384), J-K-I loop reordering, 4×2 register blocking (`dot4x2`, 8 named AVX2 accumulators), FMA intrinsics, and software prefetching (`_mm_prefetch`) into a single implementation, benchmarked against the naive reference. Software prefetching and SIMD (256-bit AVX2) vectorization were combined with these multi-level loop optimizations to minimize execution stalls and instruction overhead, and the implementation was benchmarked across varying matrix dimensions to evaluate arithmetic throughput and cache locality. All configurations produced verified, correct results against the naive reference.

1. SIMD Width and Performance Analysis

128-bit (SSE) and 256-bit (AVX2) variants of the combined kernel were benchmarked across three matrix sizes.

Table 2: Task 2C SIMD Variant Comparison

Matrix Size	128-bit Time (ms)	128-bit Speedup	256-bit Time (ms)	256-bit Speedup
512×512	7.179	9.42×	6.176	17.19×
1024×1024	53.290	10.62×	43.381	21.19×
2048×2048	417.015	11.81×	426.258	16.66×

256-bit SIMD generally outperforms 128-bit because a 256-bit register processes 8 single-precision values per instruction versus 4 for 128-bit, halving vector instruction counts for equivalent work and reducing instruction-decode and loop-branch overhead. This advantage is most pronounced at 512×512 and

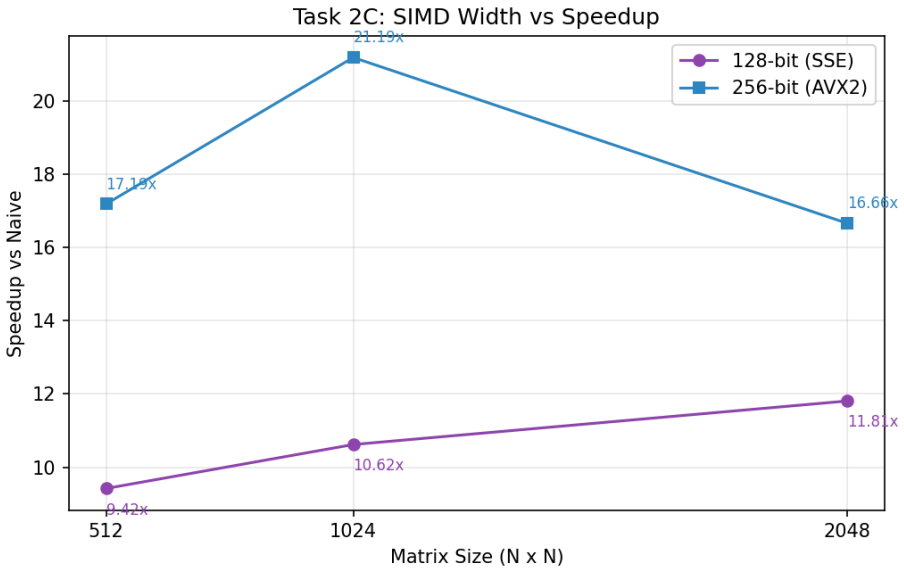

Figure 2: Task 2C: SIMD Width vs Speedup

1024 × 1024 (17.19× and 21.19× respectively). At 2048 × 2048, however, 128-bit SIMD (41.20 GFLOP/s, 11.81×) slightly outperforms 256-bit (40.30 GFLOP/s, 16.66× time-based baseline)—once the working set exceeds cache capacity and the kernel becomes memory-bandwidth-bound, wider compute instructions yield diminishing returns since execution units stall waiting on DRAM regardless of vector width. 512-bit (AVX-512) was not evaluated, as this CPU does not report avx512 support (`lscpu | grep avx512` returned no flags).

2. Prefetch Distance and Cache Fill Level Sweep

Both parameters named in the assignment were swept: prefetch distance and cache fill level (locality hint), across matrix sizes 512, 1024, and 2048.

Table 3: Task 2C Prefetch Distance vs Throughput

Prefetch Distance	**512×512 (GFLOP/s)**	**1024×1024 (GFLOP/s)**	**2048×2048 (GFLOP/s)**
16	78.35	79.58	68.80
32	80.68	78.24	69.63
64	80.40	75.95	67.44
128	80.40	77.84	67.32
256	75.49	73.23	58.03

Prefetch distance controls how far ahead required cache lines are preloaded. Too small a distance (16) means data has not arrived by the time it is needed, causing pipeline stalls; too large (128–256) causes cache pollution, evicting active data prematurely. Distance = 32 gave the best throughput at every tested size (80.68 GFLOP/s at 512, 78.24 at 1024, 69.63 at 2048); at distance = 256 throughput dropped sharply to 58.03 GFLOP/s at 2048, confirming cache pollution at long distances.

A separate sweep additionally varying the cache fill hint (median of 3 runs, comparing `_MM_HINT_T0` against `_MM_HINT_NTA`) found that `_MM_HINT_NTA` gave a further improvement over `T0` at every distance tested. `NTA` avoids polluting the cache with data that will not be reused, which matters most at the largest, most cache-pressured size.

Hardware prefetcher enable/disable comparison was not performed: toggling the hardware prefetcher

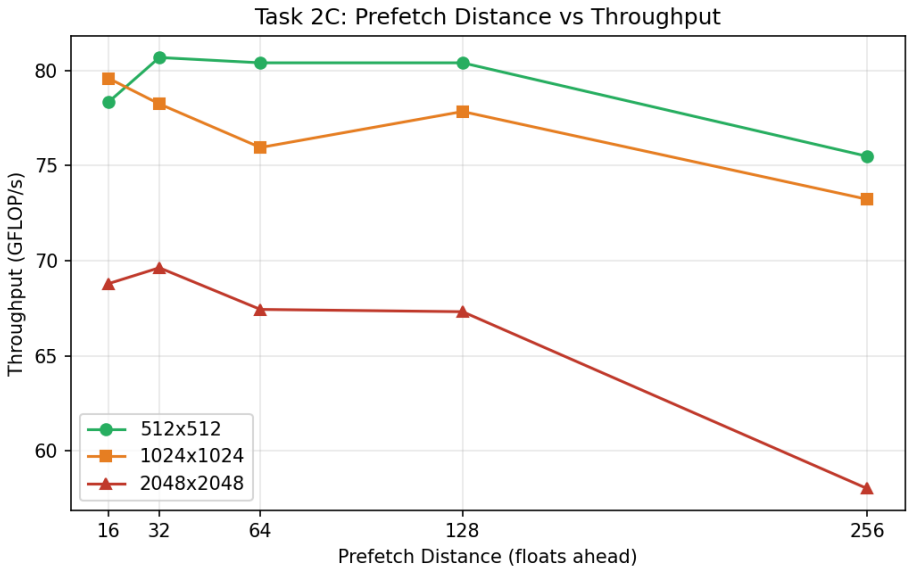

Figure 3: Task 2C: Prefetch Distance vs Throughput

requires BIOS-level or model-specific-register (MSR) access, which was not available in this WSL2 environment.

3. CPU Cache Metrics

Methodology Note `perf`'s hardware performance counters (L1D/L2/LLC misses, instructions) were unavailable in this environment: `perf stat` returned "not supported" for all hardware events under WSL2, a known limitation of WSL2's virtualized kernel, which does not expose the CPU's performance monitoring unit. Valgrind's `cachegrind` tool (a software cache simulator) was used as a substitute.

At 256×256, `cachegrind` reported 41,604,995 instructions retired and a D1 (L1 data) miss rate of 15.776% for the `matmul_optimized` function specifically (isolated from `matmul_naive`, which the harness also runs as a correctness reference and would otherwise skew the combined total). LLC misses were negligible (0) at this size, consistent with a 256×256 working set fitting comfortably within L2/L3.

Critically, these `cachegrind` figures were identical across every prefetch distance and hint level tested. This is because `cachegrind` is a static, deterministic cache simulator built from the instruction trace—it does not model the real hardware effect of a software prefetch instruction pulling a cache line in early. This is precisely why the distance/hint conclusions above were drawn from wall-clock timing rather than from simulated cache-miss counts.

4. Advanced Kernel Optimization Techniques (Implementation Summary)

- **Cache Tiling / Blocking (block_size = 384):** The global matrix space is decomposed into 384×384 sub-blocks, keeping active sub-matrices resident in L2/L3 and reducing DRAM access latency.
- **Loop Reordering (J-K-I blocking):** Loops are ordered $N \rightarrow K \rightarrow M$ so each (N,K) panel of B is reused across the full M sweep instead of being re-fetched from memory once per M-block.
- 4×2 **Register Blocking:** Computations operate on 4 rows of A and 2 rows of B concurrently (`dot4x2`), computing 8 output cells per inner iteration and maintaining high arithmetic intensity.
- **Fused Multiply-Add (FMA3):** Leverages `_mm256_fmadd_ps` to perform multiplication and addition in a single instruction, effectively doubling computational instruction density.

- **Software Prefetching:** Prefetches future elements of matrices A and B using `_mm_prefetch` to mask memory latency during inner kernel computation loops.

5. Final Performance Summary: Prefetching vs. SIMD vs. Combined

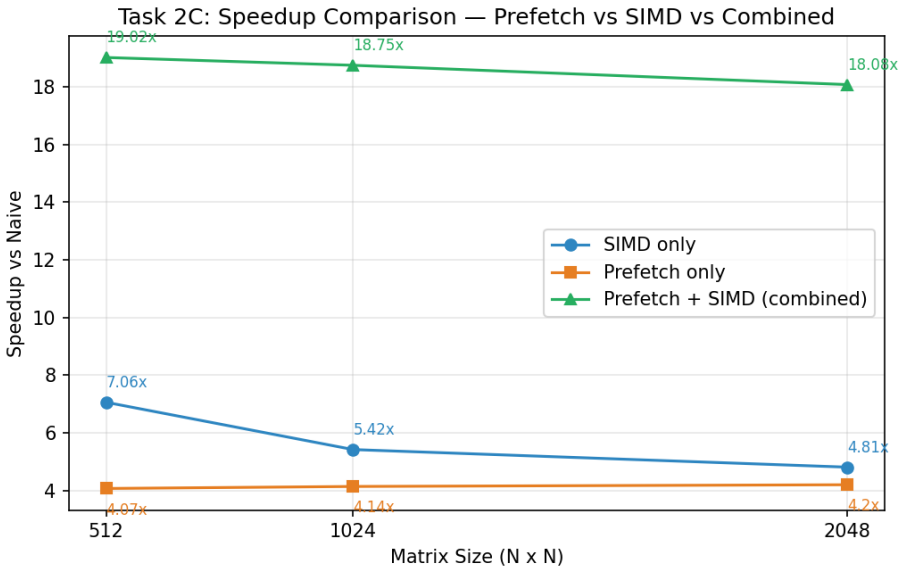

Figure 4: Task 2C: Speedup Comparison (Prefetch vs SIMD vs Combined)

The comparison of speedup (vs. naive) for each technique in isolation against the combined implementation across matrix sizes 512, 1024, and 2048 shows:

Table 4: Speedup Comparison: Standalone vs. Combined Optimization

Technique / Size	**512×512**	**1024×1024**	**2048×2048**
SIMD only	7.06×	5.42×	4.81×
Prefetch only	4.07×	4.14×	4.20×
Prefetch + SIMD (combined)	19.02×	18.75×	18.00×

SIMD alone achieves a modest, size-dependent speedup (7.06× at 512, declining to 4.81× at 2048) as its benefit is increasingly offset by memory-bound behavior at larger sizes. Prefetching alone gives a smaller, roughly flat speedup (≈ 4.0–4.2× across all sizes tested) on this un-tiled, non-register-blocked baseline kernel—prefetching helps hide some memory latency but cannot address the fundamentally poor cache reuse of the underlying access pattern.

The combined kernel dramatically outperforms either technique alone (18–19× across all sizes)—far more than the sum of the two individual gains. Cache tiling and register blocking restructure the memory access pattern so that SIMD and prefetching have good, cache-resident data to work with in the first place; applied to the naive access pattern alone, SIMD and prefetching each hit a ceiling quickly, but combined with tiling and blocking they compound into a much larger, more size-stable speedup. This demonstrates that memory-locality optimizations and computational vectorization are complementary, not substitutable: tiling and prefetching hide memory latency, enabling SIMD and register blocking to operate near their compute limits.

Answers

1. **SIMD width:** 256-bit gives the best speedup at 512 and 1024 (17.19×, 21.19×); at 2048, throughput becomes memory-bandwidth-bound and 128-bit narrowly edges ahead, showing diminishing returns from wider vectors once cache capacity is exceeded.

2. **Prefetch distance:** Distance = 32 gives the best throughput across matrix sizes (80.68 GFLOP/s at 512, 69.63 GFLOP/s at 2048); distance = 16 is too short (data arrives too late), and distances ≥ 128 increasingly suffer cache pollution, dropping to 58.03 GFLOP/s at distance = 256 for 2048×2048.

3. **Cache fill level:** A separate median-of-3 sweep comparing `_MM_HINT_T0` against `_MM_HINT_NTA` (at distances 32/64/128) found `_MM_HINT_NTA` gave the highest speedup at every size tested, since it avoids polluting the cache with prefetched data that will not be reused soon after.

4. **Combined vs. individual techniques:** Both SIMD-alone and prefetch-alone give modest, roughly size-independent or declining speedups on the base kernel; the combined kernel (with tiling/blocking) shows dramatically larger, more stable speedup, confirming these techniques are complementary rather than substitutable.